

Project Report Template

Advanced Analytics and Applications

Team 02



Authors:

Anthony Ge (7381058)

Hendrik Mehl (7410290)

???? (7354459)

Yannick Herrmann (7409728)

Contact: student@smail.uni-koeln.de

Supervisor: Univ.-Prof. Dr. Wolfgang Ketter

Co-Supervisor: Janik Muires

Chair of Information Systems for Sustainable Society
Faculty of Management, Economics and Social Sciences
University of Cologne

2026-04-01

Table of contents

1 Problem description	1
2 Data description	1
3 Data Preparation	2
3.1 Trip Data	2
3.2 Supplementary Data	2
4 Data Analytics	2
4.1 descriptive analysis	2
4.2 Analysis using SVM	2
4.3 Analysis using Neural Networks	2
4.4 Analysis using Reinforcement Learning	3
5 Conclusion	7
6 Appendix	7
7 Citations and References	11

List of Figures

1	Charging cost of one 15-minute step as a function of charging power. The exponential price multiplier makes the fastest level disproportionately expensive: 22 kW costs 9.62 dollars per step, versus 2.15 dollars at 11 kW.	3
2	Transition structure of the EV-charging MDP as one episode: from each state the agent picks a charging power (deterministic transition, reward -cost), the loop repeats for $N = 8$ steps, and at departure the drawn demand decides whether the schedule was feasible.	4
3	Learning curves of Monte Carlo control and Q-learning on one axis: expected cost of the greedy policy during training, against the exact optimum from dynamic programming.	5
4	The exact optimal policy (left) next to the greedy policies learned by Monte Carlo control and Q-learning, with the greedy charging trajectories from three starting charge levels. States with fewer than 25 training visits are grey - the agent has no reliable knowledge there.	6
5	The price of each learned decision, for both methods on a shared scale: the expected money lost by taking the learned action in a state and playing optimally afterwards. White states are exactly co-optimal choices (including ties the policy panels cannot show); grey states had too little training data.	6
6	Raw Chicago Taxi Trip Data Column Descriptions	8
7	Weather Data Columns	9
8	Boxplot of taxi trip attributes to detect outliers	10

1 Problem description

The smart mobility market is highly competitive and current reports indicate that particularly ride-hailing, using autonomous and electric vehicles will dominate the market in the near future. Traditional car manufacturers face the risk of disruption if they fail to establish competitive mobility services. Our client, a renowned German car manufacturer, seeks to enter the ride-hailing market in the US, using a fully electrified fleet. However, the client lacks operational expertise in managing electric vehicle fleets in an urban ride-hailing context. Key challenges include understanding spatial and temporal demand patterns to deploy vehicles efficiently and optimizing charging schedules to reduce energy costs while ensuring vehicles are always sufficiently charged for their working day.

To address these challenges, we analyze historical taxi trip data from Chicago as a proxy for ride-hailing demand. Specifically, we aim to conduct a descriptive analysis, which analyzes taxi demand patterns and how they vary in different spatio-temporal resolutions. Second, we build predictive models that forecast taxi trip demand for a given spatial unit and time window, enabling the client to proactively position vehicles where and when they are needed. For example, this includes estimating expected demand on a rainy Sunday morning in the city outskirts. Furthermore, we develop an intelligent charging strategy using reinforcement learning to minimize charging costs while ensuring sufficient vehicle range. Together, these analyses will enable the client to make data-driven decisions regarding fleet deployment, driver scheduling, and charging infrastructure investment.

2 Data description

To approach this task, data from multiple sources was used. The primary dataset used in this analysis is the Chicago Taxi Trips dataset (City of Chicago, 2024), which is publicly available via the Chicago Data Portal and contains taxi trips from 2024 onwards. The Taxi Trip Dataset contains one trip per row, with timestamps rounded to the nearest 15 minutes. Since the full dataset starting from 2024 contains over 16 million rows, the dataset was limited to only include trips from 2025, resulting in a raw dataset of roughly 6.8 million rows. Important attributes include trip and taxi identifiers, the timestamp of the trip, centroid coordinates of the community area or census tract where the trip took place and multiple attributes about the trips properties like price or length (Figure 6).

Furthermore, hourly Chicago weather data was obtained from the Open-Meteo API (Open-Meteo, 2026) for the period of January 2024 to January 2026. The initial dataset includes roughly 17500 entries and key variables of the dataset include temperature, precipitation and sunshine duration (Figure 7).

Points-of-Interest data for Chicago was gathered from OpenStreetMap via the Overpass API (OpenStreetMap Contributors, 2026), covering nine categories: airports, train stations, stadiums, restaurants, bars and clubs, hotels, hospitals, universities, and tourist attractions. These raw POI locations are subsequently aggregated into distance and density features for multiple spatial resolutions during data preparation.

In order to be able to analyse the data on census tract and community area level, additional geometry data containing the shapes of these areas was obtained from . These datasets contain the geometry for Chicago's 77 community areas and 878 census tracts.

3 Data Preparation

Before analysis, each dataset underwent a series of preparation steps to ensure data quality and compatibility across sources.

3.1 Trip Data

The dataset was first inspected to get an overview of the datatypes and find potential issues. This revealed that several numeric columns were incorrectly stored as strings and that the timestamp columns were not yet in datetime format. Both were corrected during preprocessing. Additionally, the dataset contained no duplicated rows or trip identifiers. Furthermore, several implausible trips were identified and removed during cleaning. These included trips with zero or negative distance (528447 trips) and trips with a duration of less than 60 seconds (381488 trips), as these trips are likely erroneous. Overlapping trips with the same taxi identifier were also removed (28320 trips) based on the assumption that these result from recording errors. Price attributes were also validated by checking for and removing trips with zero or negative fares (1455 trips) and whether there are negative tips, tolls or extras (0 trips).

The dataset was also checked for missing values, revealing that 0.18% of the non-spatial data was missing. Due to the negligible share, these rows were dropped. Because of privacy reasons, the spatial data showed many more missing values with census tract data missing for approximately 55% of trips. In these cases, the less granular community area is recorded instead, meaning spatial information is always available at either the census tract or community area level [TRUE ???].

The numeric columns were checked for outliers using boxplots (Figure 8) and removed using the z-score as a statistical solution. Observations with a z-score exceeding a threshold of 2 in any numeric column were considered outliers and removed. [REASON]. This resulted in 444102 rows, or 7.18% of the data at that point being removed, which was seen as acceptable since the remaining set still contained roughly 5.7 million rows.

[CONTINUE]

3.2 Supplementary Data

4 Data Analytics

test4

4.1 descriptive analysis

test

4.2 Analysis using SVM

test

4.3 Analysis using Neural Networks

test

4.4 Analysis using Reinforcement Learning

While the previous sections predict demand, this section turns to a decision problem at the level of a single vehicle: how should an electric vehicle (EV) be charged when the driver does not yet know how much energy the next day will require? We study this in a simulation. An EV arrives at home at 2 pm with between 5 and 15 kWh left in its 60 kWh battery and can charge for two hours, split into eight 15-minute slots. In each slot the charger runs at one of four levels (0, 7, 11, or 22 kW), matching common home wallbox and fast-charging powers. The next day's energy need is unknown while charging; we model it as normally distributed with a mean of 30 kWh and a standard deviation of 5 kWh, i.e. about half a battery of driving with realistic day-to-day variation. Charging at power p for one slot of $\Delta t = 0.25$ h costs

$$\text{cost}(p) = \alpha \cdot p \cdot \Delta t \cdot X^{p/p_{\max}},$$

with a base price of $\alpha = 0.35$ \$/kWh, a full-power price multiplier of $X = 5$, and $p_{\max} = 22$ kW. The price per kWh therefore grows exponentially with charging power, from the base rate at trickle charging up to five times the base rate at full power: one slot costs \$1.02 at 7 kW, \$2.15 at 11 kW, but \$9.62 at 22 kW (Figure 1). This reflects that fast charging is disproportionately expensive in practice, through demand charges, conversion losses, and battery wear, and it creates the central tension of the problem: charging slowly is cheap but may not fill the battery in time. Finally, if the car leaves with less energy than the day turns out to require, a flat penalty of \$100 is incurred, representing the cost and inconvenience of a stranded driver (public fast charging, a taxi, a missed appointment).

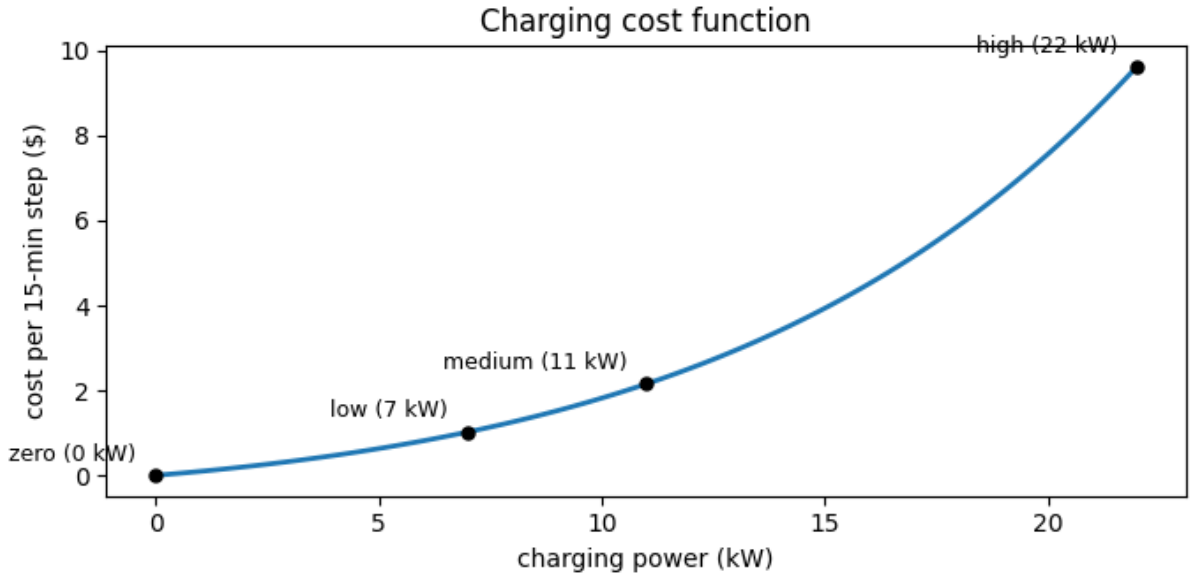


Figure 1: Charging cost of one 15-minute step as a function of charging power. The exponential price multiplier makes the fastest level disproportionately expensive: 22 kW costs 9.62 dollars per step, versus 2.15 dollars at 11 kW.

Formally, this is a Markov decision process (MDP). The state is the pair of current battery level and time slot; the actions are the four charging levels; the transitions are deterministic (each action simply adds its energy to the battery); and the reward of a step is the negative charging cost, with the \$100 penalty subtracted at departure if the realized demand exceeds the battery level. Crucially, the demand is not part of the state: the agent cannot observe it and can only hedge against it. In expectation, the hard feasibility cliff therefore becomes a smooth ramp, because the expected penalty is \$100 times the probability that demand exceeds the final charge.

Figure 2 shows one episode of this MDP: the car arrives with a random charge, the charging loop repeats eight times with deterministic transitions, and only the final departure step branches, on the drawn demand.

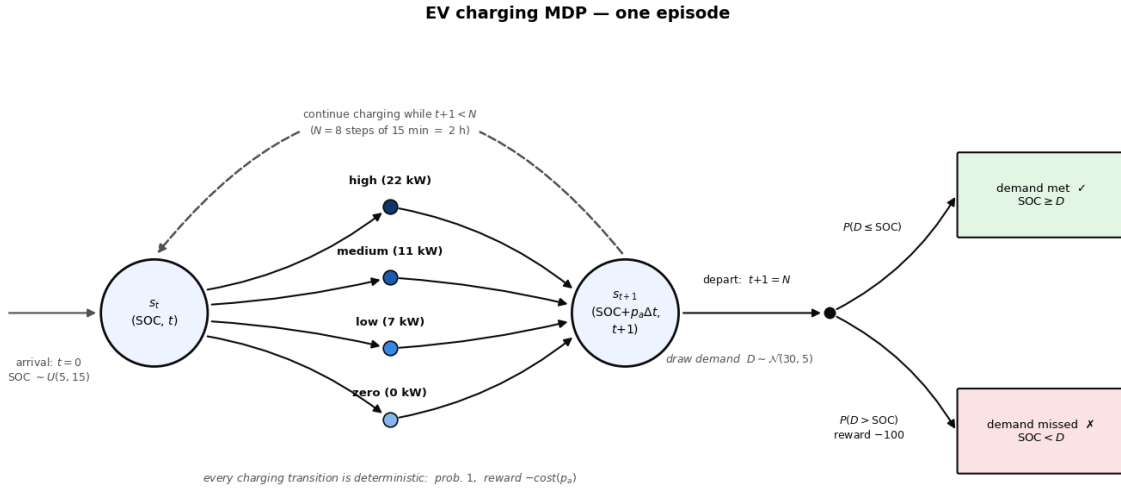


Figure 2: Transition structure of the EV-charging MDP as one episode: from each state the agent picks a charging power (deterministic transition, reward $-\text{cost}$), the loop repeats for $N = 8$ steps, and at departure the drawn demand decides whether the schedule was feasible.

Because we built the simulation ourselves, we know the complete model of this MDP: the transition rule and the demand distribution, which even gives the expected departure penalty in closed form. Whoever knows the model does not need to learn. Dynamic programming (backward induction from the departure step to arrival) computes the exact value of every action in every state, and with it the best achievable expected cost: \$39.10 per day, averaged over the random arrival charge. This is our yardstick. The learning agents, in contrast, are given none of this knowledge. They never see the tariff, the transition rule, or the demand distribution; they only observe the current state, choose an action, and receive a reward, one simulated day at a time. This is exactly the situation of a charger deployed in the real world, and it is why the agents cannot use dynamic programming and must resort to model-free reinforcement learning.

Within model-free control, the structure of the problem guides the choice of method, following standard reinforcement learning practice. The task is episodic with short episodes that always terminate after eight steps, which makes Monte Carlo control applicable and attractive: it waits until a day is over and then updates the value of each decision with the actually observed total outcome of that day, averaged over many days. Q-learning is the standard temporal-difference alternative: it updates after every single step, moving the value of a decision toward the observed step reward plus its own current estimate of the best continuation, $r + \gamma \max_{a'} Q(s', a')$. This bootstrapping is the essential difference between the two update rules: Monte Carlo uses only real outcomes (unbiased, but each day's outcome is noisy), while Q-learning partly relies on its own estimates (initially biased, but it lets the rare departure penalty propagate backwards through the day step by step). Both agents explore with an epsilon-greedy strategy, taking a random action 30 % of the time and their currently best-known action otherwise, over 80,000 simulated days. Since the state space is tiny and discrete (241 battery levels times 8 time slots, with 4 actions), both methods store their value estimates in an exact table. For the same reason we deliberately avoid a Deep Q-Network: replacing the table with a neural network is necessary only when the state space is too large or continuous to enumerate, and here it would add approximation error, hyperparameter tuning, and training instability without any benefit.

Figure 3 shows both learning curves on one axis, measured identically: the expected cost of each agent’s current best policy during training. Monte Carlo is significantly faster, reaching close to its final quality after a few thousand days, while Q-learning needs tens of thousands. The reason lies in how information travels. The decisive learning signal, the \$100 penalty, occurs only at the very end of a day. Monte Carlo hands the complete day’s outcome to every decision of that day immediately after the day ends. Q-learning instead learns from one-step lookaheads, so the penalty information must trickle backwards through the value table one time slot per visit, and each update only moves a fraction of the way (step size 0.1); many passes over the day are needed before the first charging slot feels what happens at departure.

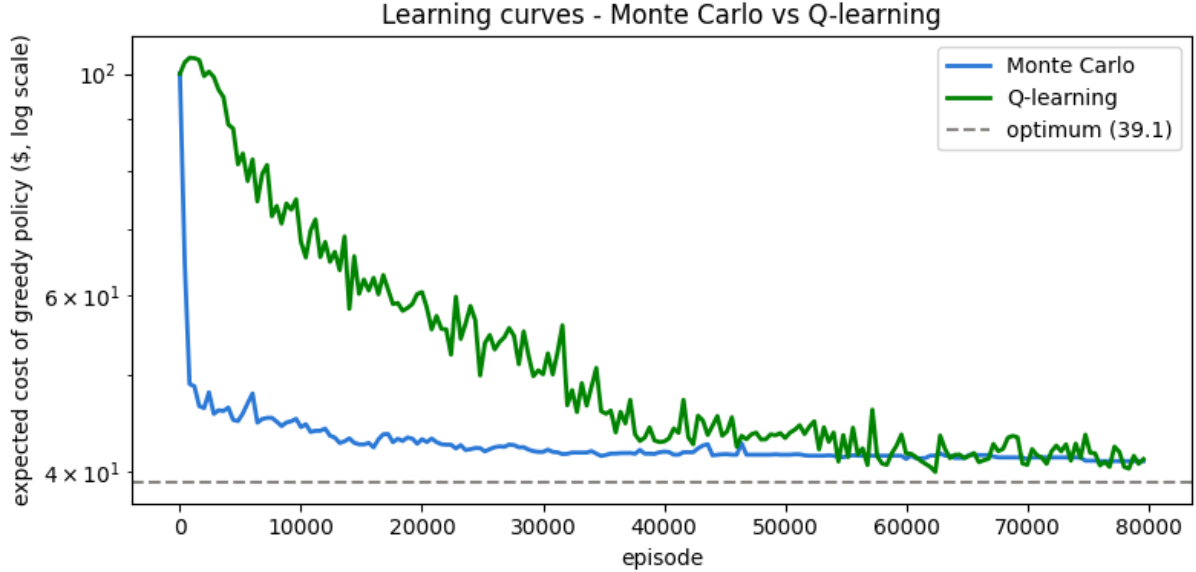


Figure 3: Learning curves of Monte Carlo control and Q-learning on one axis: expected cost of the greedy policy during training, against the exact optimum from dynamic programming.

Both agents discover the same, quite intuitive strategy as the exact optimum: charge steadily at medium power towards roughly 37 to 40 kWh, a safety buffer of about one and a half standard deviations above the average demand, and use the expensive fast-charging level only on days that start with a nearly empty battery. Charging merely to the average demand would mean running short every second day, while filling the battery completely would waste money on energy that is rarely needed; the learned buffer balances the extra charging cost against the risk of the penalty, accepting a residual shortfall probability of a few percent. In money terms, Monte Carlo reaches an expected daily cost of \$41.03 and Q-learning \$41.06, both within about 5 % of the optimal \$39.10. Figure 4 compares the learned decision rules with the optimal one across all situations (states with too little training data are shown grey), and Figure 5 prices every learned decision against the exact optimum. The learned policies look patchier than the optimal one, but this is largely harmless: the total cost depends only on how many slots of each power a schedule uses, not on their order, so different-looking choices are often exactly equally good (white in Figure 5). The decisions that genuinely cost money are rare, small, and concentrated in rarely visited states: once an agent charges sensibly, days on which it still runs short become rare, so the learning signal for fine-tuning the safety margin dries up.

So what? The practical message is that a smart charger can learn a near-optimal, risk-aware charging strategy purely from experience. It was never told the electricity tariff, the demand distribution, or the penalty; it inferred the trade-off from outcomes alone. That matters because in reality these quantities differ by household and season and are rarely written down, so a rule that adapts by itself is more deployable than one that must be re-derived for every

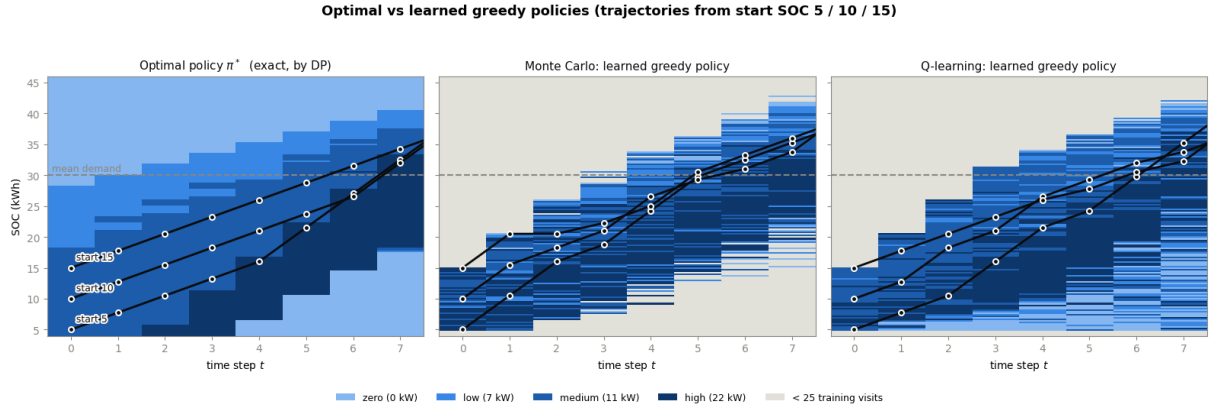


Figure 4: The exact optimal policy (left) next to the greedy policies learned by Monte Carlo control and Q-learning, with the greedy charging trajectories from three starting charge levels. States with fewer than 25 training visits are grey - the agent has no reliable knowledge there.

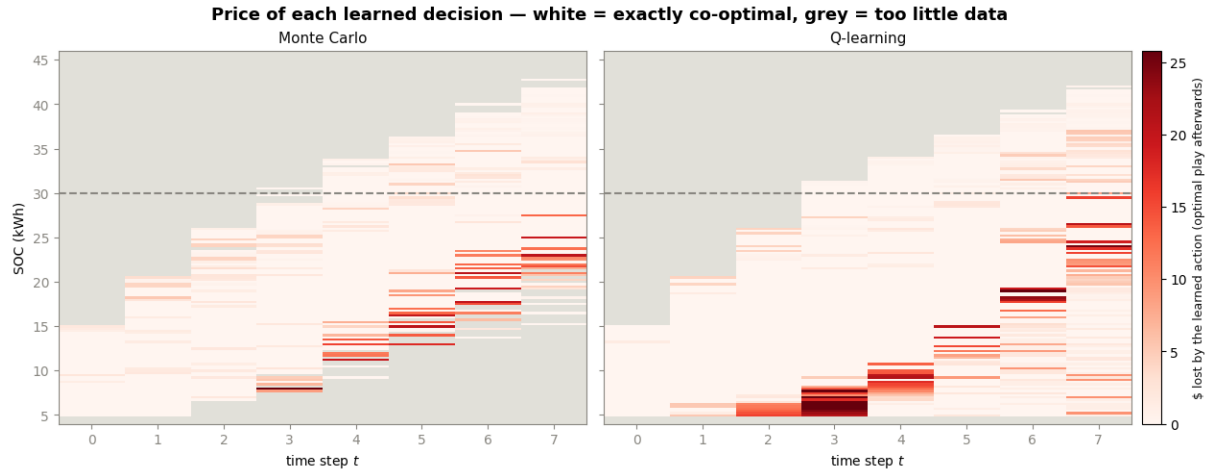


Figure 5: The price of each learned decision, for both methods on a shared scale: the expected money lost by taking the learned action in a state and playing optimally afterwards. White states are exactly co-optimal choices (including ties the policy panels cannot show); grey states had too little training data.

user. The learned behaviour also translates into simple, transferable advice: charge steadily at moderate power, keep a safety buffer above the expected need, and reserve fast charging for genuine emergencies. For the individual driver this means lower bills at near-zero risk of being stranded; at scale, fleets of chargers behaving this way flatten expensive demand peaks, a benefit for the grid that emerges from each vehicle simply minimizing its own cost.

5 Conclusion

conclusion text

6 Appendix

Columns (23)			
Column Name	Description	API Field Name	Data Type
 Trip ID	A unique identifier for the trip.	trip_id	Text
 Taxi ID	A unique identifier for the taxi.	taxi_id	Text
 Trip Start Timestamp	When the trip started, rounded to the nearest 15 minutes.	trip_start_timestamp	Floating Timestamp
 Trip End Timestamp	When the trip ended, rounded to the nearest 15 minutes.	trip_end_timestamp	Floating Timestamp
 Trip Seconds	Time of the trip in seconds.	trip_seconds	Number
 Trip Miles	Distance of the trip in miles.	trip_miles	Number
 Pickup Census Tract	The Census Tract where the trip began. For privacy, this Census Tract is not shown for some trips. This column often will be blank for locations outside Chicago.	pickup_census_tract	Text
 Dropoff Census Tract	The Census Tract where the trip ended. For privacy, this Census Tract is not shown for some trips. This column often will be blank for locations outside Chicago.	dropoff_census_tract	Text
 Pickup Community Area	The Community Area where the trip began. This column will be blank for locations outside Chicago.	pickup_community_area	Number
 Dropoff Community Area	The Community Area where the trip ended. This column will be blank for locations outside Chicago.	dropoff_community_area	Number
 Fare	The fare for the trip.	fare	Number
 Tips	The tip for the trip. Cash tips generally will not be recorded.	tips	Number
 Tolls	The tolls for the trip.	tolls	Number
 Extras	Extra charges for the trip.	extras	Number
 Trip Total	Total cost of the trip, the total of the previous columns.	trip_total	Number
 Payment Type	Type of payment for the trip.	payment_type	Text
 Company	The taxi company.	company	Text
 Pickup Centroid Latitude	The latitude of the center of the pickup census tract or the community area if the census tract has been hidden for privacy. This column often will be blank for locations outside Chicago. Read less ^	pickup_centroid_latitude	Number
 Pickup Centroid Longitude	The longitude of the center of the pickup census tract or the community area if the census tract has been hidden for privacy. This column often will be blank for locations outside Chicago. Read less ^	pickup_centroid_longitude	Number
 Pickup Centroid Location	The location of the center of the pickup census tract or the community area if the census tract has been hidden for privacy. This column often will be blank for locations outside Chicago. Read less ^	pickup_centroid_location	Point
 Dropoff Centroid Latitude	The latitude of the center of the dropoff census tract or the community area if the census tract has been hidden for privacy. This column often will be blank for locations outside Chicago. Read less ^	dropoff_centroid_latitude	Number
 Dropoff Centroid Longitude	The longitude of the center of the dropoff census tract or the community area if the census tract has been hidden for privacy. This column often will be blank for locations outside Chicago. Read less ^	dropoff_centroid_longitude	Number
 Dropoff Centroid Location	The location of the center of the dropoff census tract or the community area if the census tract has been hidden for privacy. This column often will be blank for locations outside Chicago. Read less ^	dropoff_centroid_location	Point

Figure 6: Raw Chicago Taxi Trip Data Column Descriptions

```

RangeIndex: 17568 entries, 0 to 17567
Data columns (total 15 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   date                                  17568 non-null  datetime64[ms, UTC]
1   temperature_2m                        17568 non-null  float32
2   relative_humidity_2m                 17568 non-null  float32
3   apparent_temperature                 17568 non-null  float32
4   precipitation                         17568 non-null  float32
5   rain                                 17568 non-null  float32
6   snowfall                             17568 non-null  float32
7   snow_depth                           17568 non-null  float32
8   surface_pressure                     17568 non-null  float32
9   cloud_cover                          17568 non-null  float32
10  wind_speed_10m                       17568 non-null  float32
11  wind_speed_100m                      17568 non-null  float32
12  is_day                               17568 non-null  float32
13  sunshine_duration                    17568 non-null  float32
14  direct_radiation                     17568 non-null  float32
dtypes: datetime64[ms, UTC](1), float32(14)

```

Figure 7: Weather Data Columns

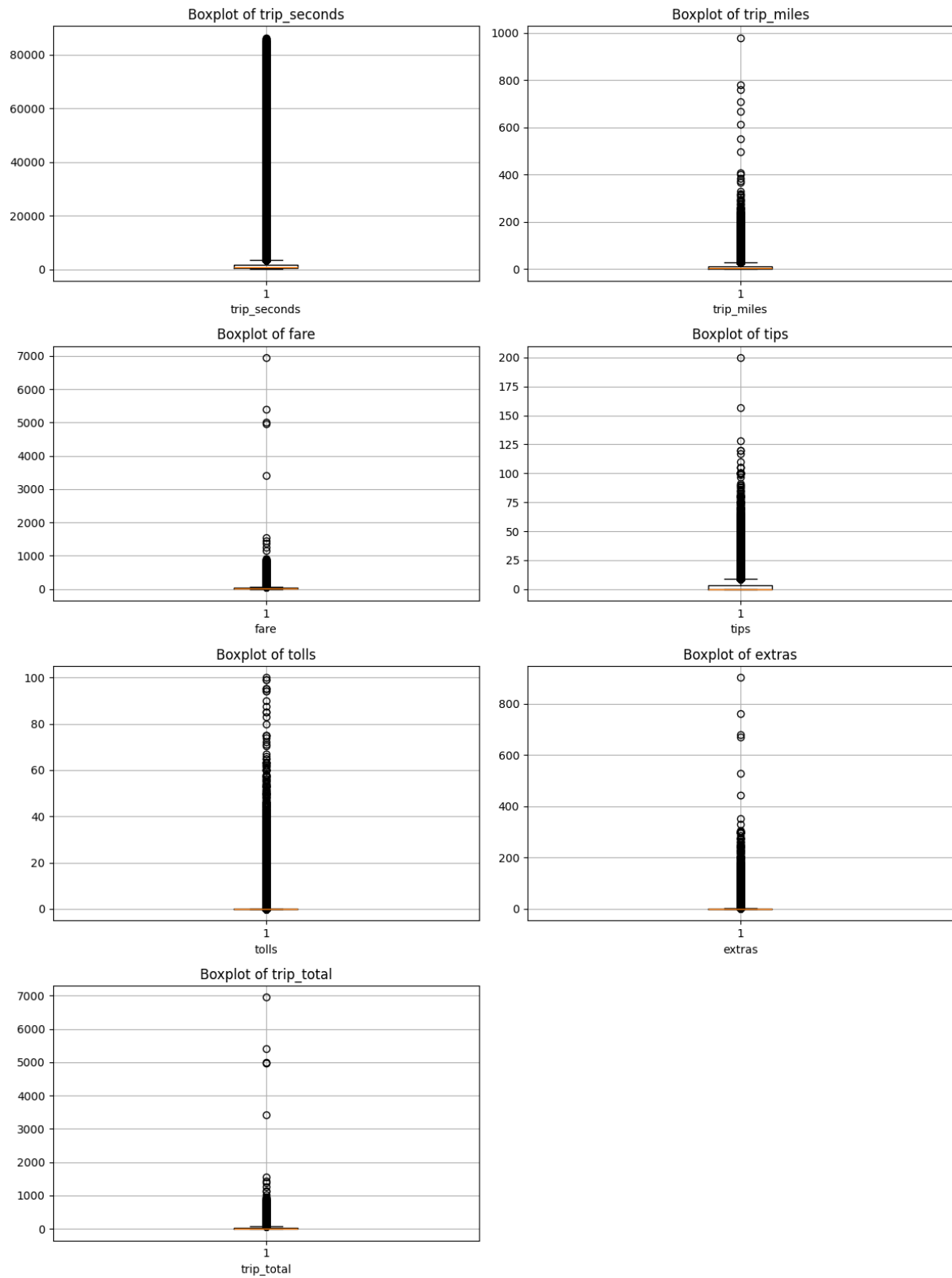


Figure 8: Boxplot of taxi trip attributes to detect outliers

7 Citations and References

You can easily cite your sources using BibLaTeX. For example, you can cite the Quarto documentation (Team, 2024) or a textbook like “R for Data Science” (Wickham et al., 2023). The bibliography will be automatically generated at the end of the document.

References

- City of Chicago. (2024). Taxi trips (2024-) [Accessed: 2026]. https://data.cityofchicago.org/Transportation/Taxi-Trips-2024-/ajtu-isnz/about_data
- Open-Meteo. (2026). Open-meteo weather api [Accessed: 28 May 2026]. <https://open-meteo.com/en/docs>
- OpenStreetMap Contributors. (2026). Overpass api — openstreetmap [Accessed: 2026]. https://wiki.openstreetmap.org/wiki/Overpass_API
- Team, Q. (2024). Quarto: An open-source scientific and technical publishing system. *Journal of Modern Data Science*. <https://quarto.org>
- Wickham, H., Çetinkaya-Rundel, M., & Grolemund, G. (2023). *R for data science*. O’Reilly Media.